

RCP103 – Rapport de TP

TP02 - Création d'un simulateur de file d'attente

Partie 1

BOURDAYA Amina, BOUTY Gabriel, CLERC Julien

11 mai 2026

Résumé

Ce TP a pour objectif de créer un programme permettant de simuler un système simple de type Client(s)/Server asynchrones. Ce premier document présente l'initialisation du projet, la schématisation du simulateur, et la description de la réalisation des premiers composants.

1 Description du système simulé

Comme évoqué en introduction, ce rapport présente la première phase de réalisation d'un simple simulateur Client(s)/Server :

Un ensemble de N clients ($N \geq 1$), envoient des messages à 1 serveur. Un portail sert d'intermédiaire entre les clients et le serveur. Compte tenu que le serveur ne peut traiter qu'un message à la fois, le portail permet de temporiser les messages en attente de traitement dans une file de taille fixe K .

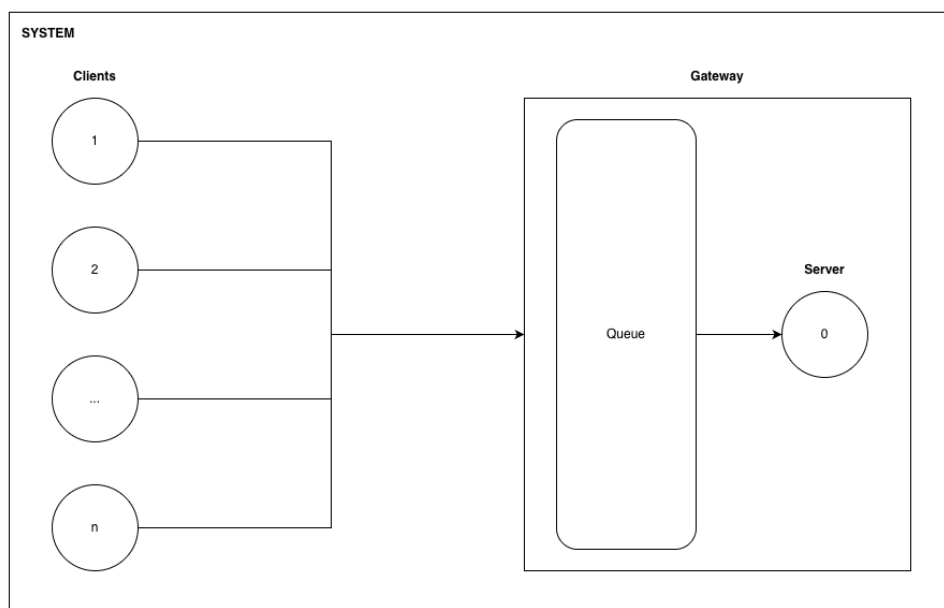


FIGURE 1 – Schéma du système simulé

Dans ce simulateur, les envois de messages suivent un processus de Poisson : le taux d'envois est constant dans le temps, les temps entre deux envois consécutifs suivent une distribution exponentielle. De même, le temps de service suit une loi exponentielle.

Compte tenu du fait que nous n'avons pour l'instant qu'un seul serveur, nous pouvons donc définir cette simulation comme un processus de file d'attente de type M/M/1/K :

- M : Arrivées selon un processus de Poisson (Markovien).
- M : Temps de service selon une distribution exponentielle (Markovien).
- 1 : Nombre de serveurs (ici, 1).
- K : Capacité du système (ici, la taille de la file du portail de capacité K).

La configuration du simulateur est définie comme suit :

- Temps moyen inter-message : configurable
- Temps de transmission : $1t$
- Temps moyen de service : configurable
- Temps total de la simulation : configurable

2 Architecture du simulateur

2.1 Architecture initiale

Afin de simuler les transmissions et traitement de messages entre le ou les *Client* et le *Server* (à travers le *Gateway*), chaque étape de gestion du *Message* seront encapsulées dans des *Event*. Ces *Event* pourront être, de fait, de 3 types différents :

- SEND : Le message est envoyé par le client.
- RECV : Le message est reçu par le portail.
- DPT : Le message est transmis au serveur (si ce dernier est disponible).

Chacuns de ces *Event* sera timestampé et stocké chronologiquement dans un ordonnanceur (*Scheduler*). Ainsi, en itérant progressivement sur les événements enregistrés dans le *Scheduler*, nous pourrons simuler l'écoulement du temps, en nous concentrant uniquement sur les échantillons temporelles qui nous intéressent.

L'ensemble de ces composants seront encapsulés dans un *Engine*, dont la boucle principale viendra interagir avec ces derniers afin de dérouler la simulation.

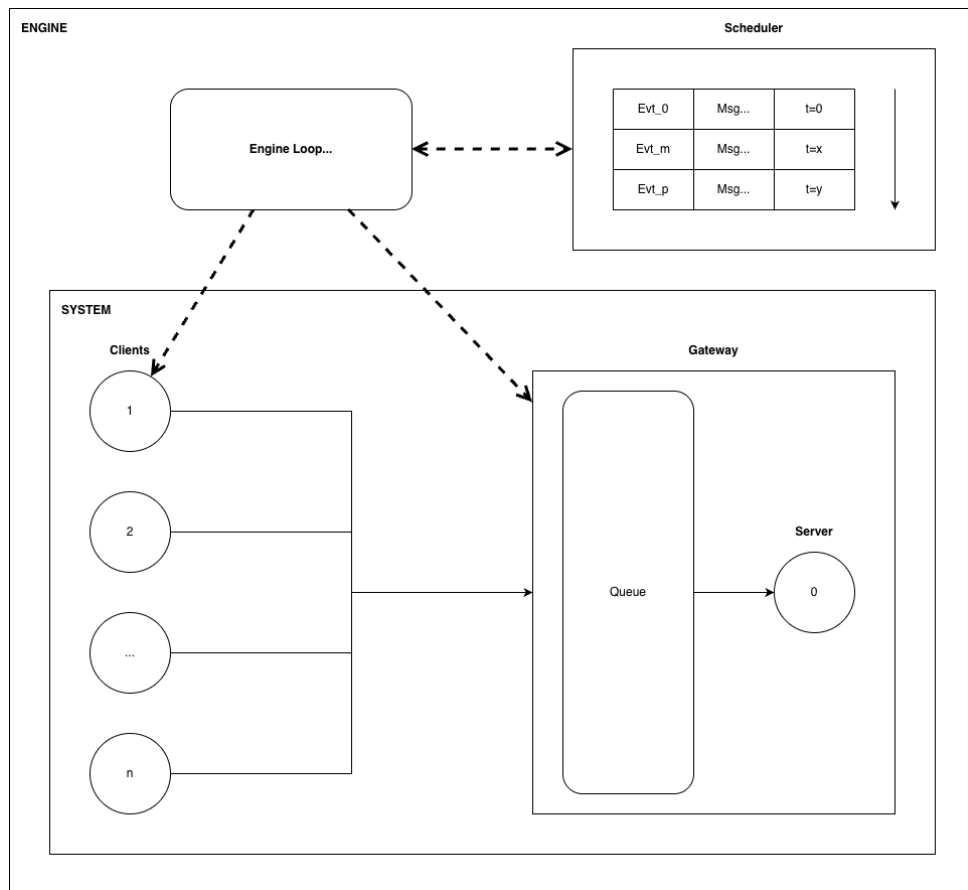


FIGURE 2 – Schéma de principe du simulateur

2.2 Organisation du code source

Le dossier fournit s'organise ainsi :

- *Root* : Le point d'entrée du programme principal (*tp02_main.py*), et les fichiers de configurations de lancement des tests
- Dossier *sources* : Le code applicatif.
- Dossier *tests* : Le code relatifs aux tests unitaires.
- Dossiers *report* et *images* : Le présent rapport les images associées.

3 Créations des premières classes

3.1 Les données : class Message et class Event

Comme évoqué précédemment, la classe *Message* est l'unité d'échange entre *Client* et *Server*. Cette classe contient :

- L'identifiant du message : `message_id`
- L'identifiant du nœud émetteur : `source`
- L'identifiant du nœud destination : `destination`
- L'instant auquel le message est envoyé par le client : `send_time`
- L'instant auquel le message arrive à la passerelle : `arrival_time`
- L'instant auquel le traitement du message commence par le serveur : `server_start`

Ce *Message* est encapsulé dans un *Event*, dont les attributs sont les suivants :

- L'identifiant de l'événement : `id`
- Le type de l'événement : `type`
- Le timestamp de l'événement : `timestamp`
- Le *Message* encapsulé : `message`

3.2 L'ordonnanceur : class Scheduler

La classe Scheduler est chargée de gérer les futurs événements du simulateur. Elle contient une liste d'événements triés par ordre chronologique.

Lorsqu'un nouvel événement est ajouté, la méthode parcourt la liste depuis la fin afin de trouver la bonne position d'insertion. Tant que le temps de l'événement déjà présent est supérieur au temps du nouvel événement, on remonte la liste. Lorsque la bonne position est trouvée, le nouvel événement est inséré avec `insert`. Cela permet de garder la liste triée sans utiliser `sort` après chaque ajout.

La méthode `pop_event` retire et retourne le premier événement de la liste. Comme la liste est maintenue triée par temps d'exécution, cet événement correspond toujours au prochain événement à traiter.

La méthode `get_current_time` retourne le temps du premier événement de la liste, c'est-à-dire le temps courant du simulateur. Si la liste est vide, la méthode retourne 0.

Enfin, la méthode `has_events` permet de savoir s'il reste encore des événements à traiter.

3.3 Les fonctions de logs : functions Trace

Le module `trace` contient trois fonctions fournissant un système de journalisation au simulateur, permettant de visualiser et d'enregistrer les événements.

Voici le prototype et la description de chaque fonction :

`def get_time_and_node(e: Event)` : prend un événement en paramètre et retourne un tuple constitué du numéro de node et du temps en fonction du type d'événement.

`def generateTraceOut(t_event: list[Event])` : prend une liste d'événements en paramètre et écrit sur la sortie standard l'ensemble des événements contenu dans la liste.

`def generateTraceCSV(t_event: list[Event])` : prend une liste d'événements en paramètre et écrit dans un fichier nommé `trace.csv` l'ensemble des événements contenu dans la liste au format CSV. Le délimiteur utilisé est le caractère `'`.

3.4 Le cœur du système : class Engine

3.5 Le point d'entrée : function main

4 Tests des premières classes

4.1 Le système de test : Pytest

Afin de vérifier le bon fonctionnement des classes implémentées, des tests unitaires ont été réalisés avec l'aide de l'utilitaire `Pytest`. L'objectif est de tester chaque classe séparément avant de l'intégrer dans le simulateur. L'ensemble des tests sont regroupés dans le répertoire `tests`.

4.2 Présentation des tests rédigés

Test de *Message* :

- Le test `test_message` vérifie que le constructeur initialise correctement les attributs du message (l'identifiant, la source et la destination, temps).
- Le test `test_message_setters` vérifie que les setters modifient correctement les valeurs internes de l'objet, après modification de l'identifiant, de la source, de la destination, et des différents temps, les getters sont utilisés pour vérifier que les nouvelles valeurs ont bien été enregistrées.
- Le test `test_message_str` vérifie la méthode d'affichage de la classe *message*. Cette méthode est utile pour le débogage, car elle permet d'afficher rapidement les informations principales d'un message.

Test de *Event* :

- Le test `test_event_init` vérifie que le constructeur initialise correctement le temps de l'événement et son type. Un événement de type `SEND_MSG` est créé avec un temps égal à 1.5, puis les getters permettent de vérifier que ces valeurs sont bien stockées.
- Le test `test_event_setters` vérifie que le temps et le type d'un événement peuvent être modifiés correctement. Par exemple, un événement initialement de type `SEND_MSG` est modifié en `RECV_MSG`, et son temps est changé de 5.5 à 8.0.
- Le test `test_event_message_content` vérifie que l'événement conserve bien le message qui lui est associé. Après avoir récupéré le message depuis l'événement, le test vérifie que son identifiant, sa source et sa destination sont corrects.
- Le test `test_event_str` vérifie la méthode d'affichage de la classe *Event*. Cette méthode affiche l'identifiant de l'événement, son type, son temps et le message associé. Elle permet donc de faciliter le suivi des événements.

Test de *Scheduler* :

- Le test `test_scheduler` vérifie que les temps des événements récupérés sont toujours dans l'ordre chronologique croissant. Cela permet de valider le rôle principal du *Scheduler* : maintenir les événements futurs triés par temps d'exécution.

Test de *Trace* :

- Le test définit par `test_get_time_and_node_event_SEND_MSG` vérifie que la fonction `get_time_and_node_event` renvoie bien le bon tuple en fonction du type de message.
- Le test définit par `test_get_time_and_node_event_RECV_MSG` vérifie que la fonction `get_time_and_node_event` renvoie bien le bon tuple en fonction du type de message.
- Le test définit par `test_get_time_and_node_event_MSG_DEPT` vérifie que la fonction `get_time_and_node_event` renvoie bien le bon tuple en fonction du type de message.
- Le test définit par `test_generateTraceOut_SEND_MSG(capsys)` vérifie que la fonction `generateTraceOut` affiche les bonnes informations en fonction du type de message.
- Le test définit par `test_generateTraceOut_RECV_MSG(capsys)` vérifie que la fonction `generateTraceOut` affiche les bonnes informations en fonction du type de message.
- Le test définit par `test_generateTraceOut_MSG_DEPT(capsys)` vérifie que la fonction `generateTraceOut` affiche les bonnes informations en fonction du type de message.
- Le test définit par `test_generateTraceCSV` vérifie que la fonction `generateTraceCSV` crée le fichier "trace.csv" avec les bonnes valeurs pour plusieurs messages.

Test de *Engine* :

- Le test `test_engine_complete` vérifie que l'exécution du `run` de *Engine* ne tombe pas dans une boucle sans fin, et que l'ensemble des *Message* et *Event* sont bien consommés à l'issus de l'exécution.

4.3 Exécution des tests

Les tests peuvent être lancés depuis le répertoire principal en tapant la commande `'pytest'` (attention, `Pytest` doit être préalablement installé). L'ensemble des tests présents dans le répertoire `tests` seront exécutés.

5 Conclusion